Project Report
32-bit Single-Cycle MIPS Processor

Prepared by: Abdalhalim Shokry Zaki

Junior Electronics and Communication Engineering

Institution: Faculty of Engineering, Ain Shams University

Date: May 2026

Repository: https://github.com/AbdalhalimShokry/Single-Cycle-MIPS

Table of Contents

Abstract

This project presents the design and implementation of a 32-bit single-cycle MIPS processor using Verilog HDL. The work represents an essential first step into digital system design, RTL (Register Transfer Level) architecture development, and hardware verification methodologies. The processor is based on the classical MIPS architecture and supports a subset of arithmetic, logical, memory-access, branching, and jump instructions.

1.0 Introduction

1.1 Background on MIPS Architecture

The MIPS architecture is one of the most widely studied Reduced Instruction Set Computer (RISC) architectures in computer engineering and digital system design. MIPS processors are designed around the principle of simplifying instruction execution by using a limited and optimized instruction set. This simplification allows instructions to execute efficiently while maintaining a clear and organized hardware structure.

Unlike Complex Instruction Set Computer (CISC) architectures, RISC architectures such as MIPS rely on simple instructions with fixed formats and predictable execution behavior. Most MIPS instructions are executed in a small number of clock cycles and use a load/store architecture, where arithmetic and logical operations are performed only on registers while memory access operations are restricted to dedicated load and store instructions.

The single-cycle implementation of the MIPS processor executes each instruction completely within one clock cycle. All stages of instruction execution, including instruction fetch, decode, execution, memory access, and write-back, are completed before the next clock edge. Although this architecture is not optimized for high performance, it provides a clear understanding of processor datapath organization, control logic generation, and hardware module interaction.

1.2 Project Objectives

The primary objective of this project is to design and implement a functional 32-bit single-cycle MIPS processor using Verilog HDL while developing a comprehensive understanding of datapath organization, control signal generation, and the interaction between hardware modules during instruction execution within a single clock cycle. The processor is developed using a modular design methodology in which each hardware component is implemented and verified independently before full system-level integration. Through this implementation, the project also strengthens practical knowledge in computer architecture, digital logic design, simulation, debugging, and modular hardware development techniques commonly utilized in modern FPGA and ASIC design flows.

The project aims to achieve the following objectives:

- Design a complete single-cycle datapath capable of executing multiple MIPS instruction formats
- Implement major processor components including the Program Counter, Instruction Memory, Register File, ALU, Data Memory, Control Unit, and multiplexers
- Support arithmetic, logical, branching, memory access, and jump instructions
- Generate proper control signals for datapath operation using a centralized control unit
- Implement fault detection mechanisms for invalid memory access and unsupported instructions
- Validate processor functionality through simulation and waveform analysis
- Develop a scalable architecture that can later be extended toward pipelined processor implementation

The project also focuses on improving understanding of RTL design techniques, combinational and sequential logic implementation, modular hardware development, and processor-level debugging methodologies.

2.0 Project Specifications

The implemented processor follows a 32-bit MIPS single-cycle architecture. The processor executes one complete instruction during each clock cycle using a unified datapath controlled by combinational control logic.

The processor supports the following instruction categories:

R-Type Instructions

- ADD
- SUB
- AND
- OR
- XOR
- SLT

I-Type Instructions

- LW
- SW
- ADDI
- BEQ
- BNE

J-Type Instructions

- J
    - **It directly generates the target address by concatenating:**
        - The upper 4 bits of PC + 4
        - The 26-bit address field from the instruction
        - Two appended zero bits for word alignment

The processor uses:

- 32-bit instruction width
- 32-bit datapath
- 32 general-purpose registers
- Word-addressable instruction and data memory
- Separate instruction and data memories
- Synchronous register file and data memory write operations
- Asynchronous register file and memory read operations

The design also includes:

- Overflow detection in arithmetic operations
- Illegal instruction detection
- Invalid memory access detection
- Processor halt mechanism during critical faults

The processor is implemented entirely using structural and behavioral Verilog HDL modules.

3.0 Design Methodology

3.1 Overview of the Single-Cycle Architecture

The processor architecture is divided into several interconnected hardware modules that collectively perform instruction execution within a single clock cycle. The datapath begins with instruction fetching from instruction memory using the Program Counter value. The fetched instruction is then decoded by the Control Unit, which generates the required control signals for the datapath.

Operands are retrieved from the Register File and routed into the ALU, where arithmetic or logical operations are performed. For memory-related instructions, the ALU generates memory addresses used by the Data Memory module. Finally, the resulting data is written back into the Register File when required.

The architecture uses multiplexers extensively to dynamically select datapath routes depending on the instruction type. Branch and jump instructions modify the Program Counter logic to enable non-sequential execution flow.

The design methodology focused on:

- Modular implementation
- Independent module verification
- Hierarchical integration
- Incremental testing
- Simplified debugging and scalability

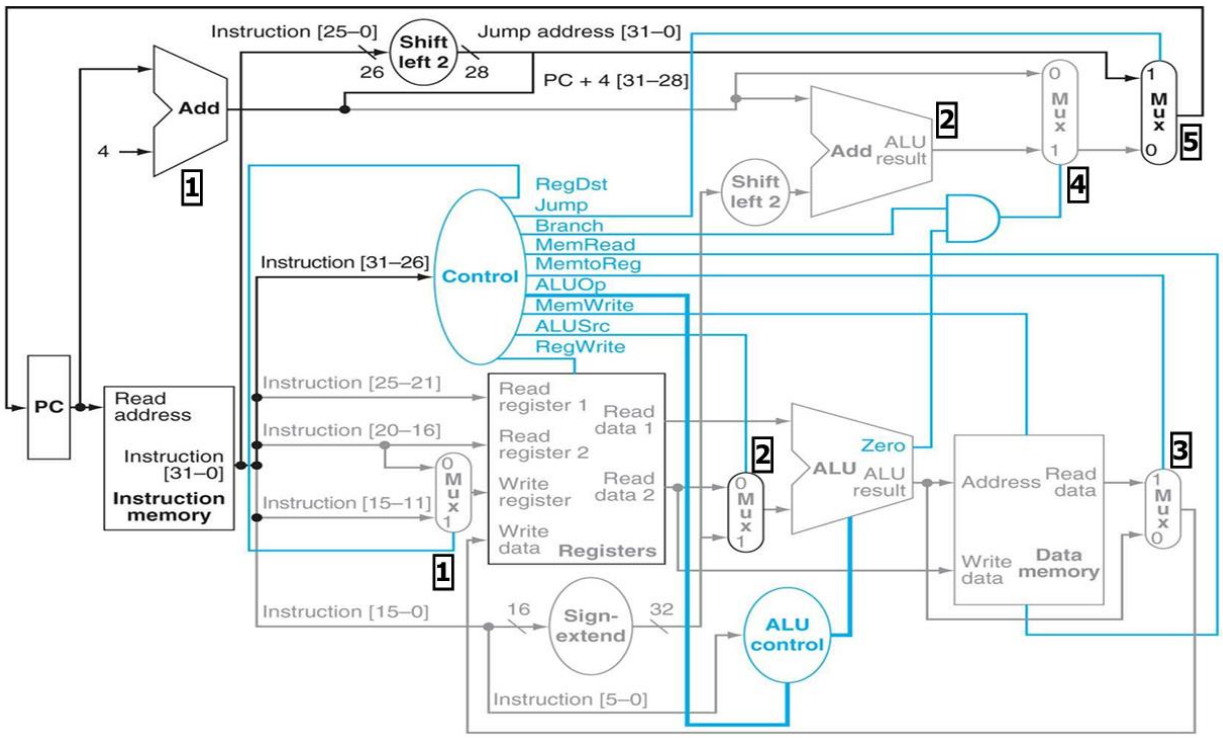

Figure 1: Single-Cycle MIPS Data Path

3.2 Program Counter and Next-State Logic

The Program Counter (PC) is responsible for storing the address of the current instruction being executed. During normal operation, the PC value increments by four bytes to point to the next instruction in memory.

For branch and jump instructions, the next PC value is determined using multiplexing logic controlled by branch and jump signals. Branch instructions use ALU comparison results combined with branch control signals to determine whether the branch target address should replace the sequential PC value.

The Program Counter module supports processor halting when critical faults occur, including:

- Arithmetic overflow
- Illegal instructions
- Invalid data memory access

Program_Counter.v

```
module Program_Counter(
    input [31:0] next_pc,
    input clk, rst_n,
    input overflow_flag, alu_ctrl_fault_flag,
    input data_mem_fault_flag, ctrl_unit_fault_flag,
    output reg [31:0] current_pc
    );

    wire halt;
    assign halt = (overflow_flag | alu_ctrl_fault_flag | ctrl_unit_fault_flag | data_mem_fault_flag);

    always @(posedge clk or negedge rst_n)
        begin
            if(!rst_n)
                begin
                    current_pc <= 32'b0;
                end
            else if(!halt)
                begin
                    current_pc <= next_pc;
                end
        end
endmodule
```

Figure 2: Program Counter

3.3 Instruction Memory Design

The Instruction Memory module stores the machine instructions executed by the processor. The memory is organized as a 32-bit wide memory array capable of storing multiple instructions.

The Program Counter output is converted from byte addressing into word addressing by shifting the address right by two bits.

Invalid memory accesses are detected using boundary checking logic. If an invalid instruction address is detected, the memory outputs a NOP-equivalent instruction to prevent unintended processor behavior.

Instruction_Memory.v

```
module Instruction_Memory(
    input [31:0] current_pc,
    output instr_mem_fault_flag,
    output [31:0] instruction
    );

    reg [31:0] memory_reg [0:255]; // Memory stores up to 256 instructions each of 32-bit length
    wire [31:0] mem_address; // Byte address -> Word address
    assign mem_address = current_pc >> 2;

    /* Indicates invalid instruction memory access (out-of-bounds), can be used furthur in error detection and handling
       It does not stop the CPU like other faults */
    assign instr_mem_fault_flag = (mem_address >= 32'd256);

    // For invalid access, instruction = 32'd0 which means no operation (nop) till the accessing become valid again
    assign instruction = (!instr_mem_fault_flag)? memory_reg[mem_address] : 32'd0;
endmodule
```

Figure 3: Instruction Memory

3.4 Register File Design

The Register File serves as the primary storage location for operands and computation results during processor execution. It contains 32 general-purpose registers, each with a width of 32 bits. Two registers can be read simultaneously while one register can be written during the same clock cycle.

The Register File supports:

- Two asynchronous read ports
- One synchronous write port
- Register zero protection
- Reset initialization

Register $0 is permanently fixed to zero and cannot be modified. Write operations occur only on the positive edge of the clock when the write-enable signal is active.

Register_File.v

```
module Reg_File(
    input [4:0] read_reg_1, read_reg_2, write_reg,
    input [31:0] reg_file_write_data,
    input reg_write,
    input clk, rst_n,
    output [31:0] reg_file_read_data_1, reg_file_read_data_2
    );

    reg [31:0] registers [0:31]; // 32 registers each of 32-bit length
    integer i; // Used as index in for loop

    // Synchronous write
    always @(posedge clk or negedge rst_n)
        begin
            if(!rst_n)
            begin
                for(i = 1; i < 32; i = i + 1)
                    begin
                        registers[i] <= 32'd0;
                    end
            end
            else if(reg_write && (write_reg != 5'd0))
                begin
                    registers[write_reg] <= reg_file_write_data;
                end
        end

    // Asynchronous read - Always on
    assign reg_file_read_data_1 = (read_reg_1 == 5'd0)? 32'd0 : registers[read_reg_1];
    assign reg_file_read_data_2 = (read_reg_2 == 5'd0)? 32'd0 : registers[read_reg_2];
endmodule
```

Figure 4: Register File

3.5 Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations required by processor instructions.

Supported operations include:

- Addition
- Subtraction
- Bitwise AND
- Bitwise OR
- Bitwise XOR
- Set-on-less-than (SLT)

The ALU generates:

- Computation result
- Zero flag
- Overflow flag

The zero flag is primarily used for branch decision logic, while the overflow flag is used for fault detection during signed arithmetic operations and is implemented explicitly for both addition and subtraction operations.

ALU.v

```
module ALU (
    input signed [31:0] alu_input_1, alu_input_2,
    input [2:0] alu_sel,
    output zero_flag,
    output reg overflow_flag,
    output reg signed [31:0] alu_res
    );

    always @(*)
        begin
            overflow_flag = 1'b0;
            alu_res = 32'd0;

            case(alu_sel)
                3'b000: // ADD
                    begin
                        alu_res = alu_input_1 + alu_input_2;
                        // For ADD, if sign of 1st argument = sign of 2nd argument ≠ sign of res => Overflow
                        overflow_flag = ((alu_input_1[31] & alu_input_2[31] & ~alu_res[31]) | (~alu_input_1[31] & ~alu_input_2[31] & alu_res[31]));
                    end
                3'b001: // SUB
                    begin
                        alu_res = alu_input_1 - alu_input_2;
                        // For SUB, if sign of 1st argument ≠ sign of 2nd argument = sign of res => Overflow
                        overflow_flag = ((alu_input_1[31] & ~alu_input_2[31] & ~alu_res[31]) | (~alu_input_1[31] & alu_input_2[31] & alu_res[31]));
                    end
                3'b010: alu_res = alu_input_1 & alu_input_2;
                3'b011: alu_res = alu_input_1 | alu_input_2;
                3'b100: alu_res = alu_input_1 ^ alu_input_2;
                3'b101: alu_res = ($signed(alu_input_1) < $signed(alu_input_2)) ? 32'd1 : 32'd0; // Set on less than (SLT)
                default: alu_res = 32'd0;
            endcase
        end
        assign zero_flag = (alu_res == 32'd0);
endmodule
```

Figure 5: ALU

3.6 ALU Control Unit

The ALU Control Unit translates high-level ALU operation codes generated by the main Control Unit into detailed ALU selection signals.

For R-type instructions, the ALU Control Unit decodes the instruction function field to determine the required ALU operation. Unsupported function codes generate fault signals to prevent illegal execution behavior.

The separation between the main Control Unit and the ALU Control Unit simplifies control logic organization and improves modularity.

ALU_Control.v

```
module ALU_Control(
    input [5:0] funct,
    input [1:0] alu_op,
    output reg alu_ctrl_fault_flag,
    output reg [2:0] alu_sel
    );

    always @(*)
        begin
            alu_ctrl_fault_flag = 1'b0;
            alu_sel = 3'b000;

            case(alu_op)
                2'b00: alu_sel = 3'b000; // ADD
                2'b01: alu_sel = 3'b001; // SUB
                2'b10: // Depend on function field (R-Type)
                    begin
                        case(funct)
                            6'd32: alu_sel = 3'b000; // ADD
                            6'd34: alu_sel = 3'b001; // SUB
                            6'd36: alu_sel = 3'b010; // AND
                            6'd37: alu_sel = 3'b011; // OR
                            6'd38: alu_sel = 3'b100; // XOR
                            6'd42: alu_sel = 3'b101; // SLT
                            default: alu_ctrl_fault_flag = 1'b1; // To indicate unused instruction
                        endcase
                    end
                default: alu_ctrl_fault_flag = 1'b1; // To indicate unused instruction
            endcase
        end
endmodule
```

Figure 6: ALU Control Unit

3.7 Control Unit Design

The Control Unit generates all datapath control signals based on the instruction opcode field.

The generated control signals include:

- Register destination selection
- ALU source selection
- Memory read enable
- Memory write enable
- Register write enable
- Branch control
- Jump control
- Memory-to-register selection
- ALU operation code

Unsupported instructions activate a fault flag used to halt processor execution safely.

Control.v

```
module Control_Unit(
    input [5:0] control_instruction,
    output reg ctrl_unit_fault_flag, // Used to prevent any unused/illegal instructions
    output reg reg_dst, jump, branch_equal, branch_not_equal, mem_read,
    output reg  mem_to_reg, mem_write, alu_src, reg_write,
    output reg [1:0] alu_op
    );

    always @(*)
    begin
        ctrl_unit_fault_flag = 1'b0;

        reg_dst             = 1'b0;
        jump                = 1'b0;
        branch_equal        = 1'b0;
        branch_not_equal    = 1'b0;
        mem_read            = 1'b0;
        mem_to_reg          = 1'b0;
        mem_write           = 1'b0;
        alu_src             = 1'b0;
        reg_write           = 1'b0;
        alu_op              = 2'b00;

        case(control_instruction)
            6'b000000: // R-Format Instructions
                begin
                    reg_dst = 1'b1;
                    reg_write = 1'b1;
                    alu_op = 2'b10;
                end
```

```
                6'b100011: // Load Word (LW)
                    begin
                        alu_src = 1'b1;
                        mem_read = 1'b1;
                        mem_to_reg = 1'b1;
                        reg_write = 1'b1;
                        alu_op = 2'b00;
                    end
                6'b101011: // Store Word (SW)
                    begin
                        alu_src = 1'b1;
                        mem_write = 1'b1;
                        alu_op = 2'b00;
                    end
                6'b000100: // Branch if Equal (BEQ)
                    begin
                        branch_equal = 1'b1;
                        alu_op = 2'b01;
                    end
                6'b000101: // Branch if not Equal (BNE)
                begin
                    branch_not_equal = 1'b1;
                    alu_op = 2'b01;
                end
                6'b000010: // Jump
                    begin
                        jump = 1'b1;
                    end
```

```
            6'b001000: // ADDI (Add Immediate)
                begin
                    alu_src = 1'b1;
                    reg_write = 1'b1;
                    alu_op = 2'b00; // It is like normal add but with an immediate
                end
            default: ctrl_unit_fault_flag = 1'b1; // Unused/illegal instruction
        endcase
    end
endmodule
```

Figure 7: Control Unit

3.8 Data Memory Design

The Data Memory module is responsible for handling load and store instructions. Memory addresses generated by the ALU are converted from byte addresses into word addresses before accessing the memory array.

The memory supports:

- Synchronous write operations
- Asynchronous read operations
- Boundary checking
- Fault detection

Write operations occur only during active memory write control signals and valid address ranges. Invalid memory access attempts activate a fault flag that can halt processor operation.

The design intentionally omits memory reset initialization to simplify synthesis and maintain realistic memory behavior.

Data_Memory.v

```
module Data_Memory(
    input [31:0] address, write_data,
    input mem_write, mem_read, clk,
    output data_mem_fault_flag, // Used to indicate invalid memory access attempt
    output reg [31:0] data_mem_read_data
    );

    reg [31:0] memory_reg [0:1023]; // Memory array
    wire [31:0] mem_address; // Byte address -> Word address
    assign mem_address = address >> 2;

    // Synchronous write
    always @(posedge clk)
        begin
            if(mem_write && (mem_address < 32'd1024))
                begin
                    memory_reg[mem_address] <= write_data;
                end
        end

    // Asynchronous read - Controlled
    always @(*)
        begin
            data_mem_read_data = 32'd0;

            if(mem_read && (mem_address < 32'd1024))
                begin
                    data_mem_read_data = memory_reg[mem_address];
                end
        end

    // If reading or writing are on and the address at this moment exceeds memory addressing limit the flag should turn on
    assign data_mem_fault_flag = ((mem_address >= 32'd1024) && (mem_read || mem_write));
endmodule
```

Figure 8: Data Memory

3.9 Sign Extension

Immediate values extracted from instructions are extended from 16 bits to 32 bits using sign extension logic. This allows signed arithmetic operations and branch offset calculations to function correctly.

Sign_Extender.v

```
module Sign_Extender(
    input [15:0] sign_extender_input,
    output [31:0] sign_extender_output
    );

    assign sign_extender_output = {{16{sign_extender_input[15]}}, sign_extender_input};
endmodule
```

Figure 9: Sign Extender

3.10 Multiplexer-Based Datapath Routing

Major multiplexers include:

- Register destination selector
- ALU operand selector
- Write-back data selector
- Branch target selector
- Program Counter source selector

MUX.v

```
module MUX2x1(
    input [31:0] mux_input_1, mux_input_2,
    input mux_sel,
    output [31:0] mux_out
    );

    assign mux_out = mux_sel? mux_input_2 : mux_input_1;
endmodule
```

Figure 10: 2-to-1 MUX

3.11 Adder

Adder.v

```
module Adder(
    input [31:0] adder_input_1, adder_input_2,
    output [31:0] adder_output
    );

    assign adder_output = adder_input_1 + adder_input_2;
endmodule
```

Figure 11: Adder

3.12 Shift Left By 2

Shift_Left_2.v

```
module Shift_Left_2(
    input [31:0] Shift_Left_2_input,
    output [31:0] Shift_Left_2_output
    );

    assign Shift_Left_2_output = Shift_Left_2_input << 2;
endmodule
```

Figure 12: Shift Left By 2

4.0 Top Module Hierarchy and Integration

The processor is implemented using a hierarchical modular design approach. Each hardware component is developed as an independent Verilog module and later integrated into the top-level processor module.

The hierarchy improves:

- Readability
- Debugging efficiency
- Reusability
- Scalability
- Verification simplicity

The Top Module interconnects all datapath and control modules to form the complete processor architecture.

It manages all interconnections between:

- Program Counter
- Instruction Memory
- Register File
- ALU
- Data Memory
- Control Units
- Adders
- Multiplexers
- Sign Extender

Top_Module.v

```v
`timescale 1ns / 1ps
module Top_Module(
    input clk, rst_n
    );

    // Wires Declaration

    // Program Counter (PC) I/P
    wire [31:0] next_pc;

    // Program Counter (PC) O/Ps
    wire [31:0] current_pc; // Also Instruction Memory I/P
    wire [31:0] pc_by_4; // current_pc + 32'd4

    // Instruction Memory (IM) O/Ps
    wire instr_mem_fault_flag; // Can be used in future adjustments and detection
    wire [31:0] instruction; // Also I/P of some Modules

    wire [31:0] jump_address;
    assign jump_address = {pc_by_4[31:28], instruction[25:0], 2'b00};

    wire [15:0] immediate_field; // Used in I-Format instructions
    assign immediate_field = instruction[15:0];

    wire [31:0] immediate_field_extended; // MUXed with read_data_2 to get ALU 2nd input
    wire [31:0] immediate_field_extended_SL2; // immediate_field_extended Shifted Left By 2

    // Register File (RF) I/Ps
    wire [4:0] read_reg_1;
    assign read_reg_1 = instruction[25:21];

    wire [4:0] read_reg_2;
    assign read_reg_2 = instruction[20:16];

    wire [4:0] write_reg_input_2;
    assign write_reg_input_2 = instruction[15:11]; // MUXed with read_reg_2 to get Write Register

```

```
    wire [4:0] write_reg; // O/P of MUXing read_reg_2 & write_reg_input_2
    wire [31:0] reg_file_write_data;

    // Register File (RF) O/Ps
    wire [31:0] reg_file_read_data_1; // Also ALU 1st I/P
    wire [31:0] reg_file_read_data_2; // Also I/P of DM Write Data

    // Control Unit (CU) I/P
    wire [5:0] control_instruction;
    assign control_instruction = instruction[31:26];

    // Control Unit (CU) O/P (Control Signals)
    wire ctrl_unit_fault_flag;
    wire reg_dst, jump, branch_equal, branch_not_equal, mem_read;
    wire mem_to_reg, mem_write, alu_src, reg_write;
    wire [1:0] alu_op;

    wire branch_sel; // Result of ANDing Branch Control Signals with ALU Zero Flag
    assign branch_sel = ((branch_equal & zero_flag) | (branch_not_equal & ~zero_flag));

    // ALU Control I/Ps
    wire [5:0] funct;
    assign funct = instruction[5:0];

    // ALU Control O/Ps
    wire alu_ctrl_fault_flag;
    wire [2:0] alu_sel;

    // ALU I/P
    wire [31:0] alu_input_2;

    // ALU O/Ps
    wire zero_flag;
    wire overflow_flag;
    wire [31:0] alu_res; // Also I/P of DM Address & MUXed with DM Read Data to get RF Write Data
```

```

     // Data Memory (DM) O/Ps
     wire data_mem_fault_flag;
     wire [31:0] data_mem_read_data;

     // Internal Wires
     wire [31:0] branch_pc_by_4; // O/P of Adding pc_by_4 and immediate_field_extended_SL2
     wire[31:0] branch_jump; // O/P of Muxing branch_pc_by_4 and pc_by_4

     // ------------------------------------------------------------------------------

```

```
84      // Modules
85
86      Program_Counter PC(
87          .next_pc(next_pc),
88          .clk(clk),
89          .rst_n(rst_n),
90          .overflow_flag(overflow_flag),
91          .alu_ctrl_fault_flag(alu_ctrl_fault_flag),
92          .data_mem_fault_flag(data_mem_fault_flag),
93          .ctrl_unit_fault_flag(ctrl_unit_fault_flag),
94          .current_pc(current_pc)
95      );
96
97      Instruction_Memory IM(
98          .current_pc(current_pc),
99          .instr_mem_fault_flag(instr_mem_fault_flag),
100         .instruction(instruction)
101     );
102
103     Reg_File RF(
104         .read_reg_1(read_reg_1),
105         .read_reg_2(read_reg_2),
106         .write_reg(write_reg),
107         .reg_file_write_data(reg_file_write_data),
108         .reg_write(reg_write),
109         .clk(clk),
110         .rst_n(rst_n),
111         .reg_file_read_data_1(reg_file_read_data_1),
112         .reg_file_read_data_2(reg_file_read_data_2)
113     );
114
```

```
115     Control_Unit CU(
116         .control_instruction(control_instruction),
117         .ctrl_unit_fault_flag(ctrl_unit_fault_flag),
118         .reg_dst(reg_dst),
119         .jump(jump),
120         .branch_equal(branch_equal),
121         .branch_not_equal(branch_not_equal),
122         .mem_read(mem_read),
123         .mem_to_reg(mem_to_reg),
124         .mem_write(mem_write),
125         .alu_src(alu_src),
126         .reg_write(reg_write),
127         .alu_op(alu_op)
128     );
129 
130     ALU_Control ALU_Ctrl(
131         .funct(funct),
132         .alu_op(alu_op),
133         .alu_ctrl_fault_flag(alu_ctrl_fault_flag),
134         .alu_sel(alu_sel)
135     );
136 
137     ALU ALU(
138         .alu_input_1(reg_file_read_data_1),
139         .alu_input_2(alu_input_2),
140         .alu_sel(alu_sel),
141         .zero_flag(zero_flag),
142         .overflow_flag(overflow_flag),
143         .alu_res(alu_res)
144     );
145 
```

```
146     Data_Memory DM(
147         .address(alu_res),
148         .write_data(reg_file_read_data_2),
149         .mem_write(mem_write),
150         .mem_read(mem_read),
151         .clk(clk),
152         .data_mem_fault_flag(data_mem_fault_flag),
153         .data_mem_read_data(data_mem_read_data)
154     );
155 
156     Sign_Extender Immediate_Extender(
157         .sign_extender_input(immediate_field),
158         .sign_extender_output(immediate_field_extended)
159     );
160 
161     Shift_Left_2 Immediate_Shifting(
162         .Shift_Left_2_input(immediate_field_extended),
163         .Shift_Left_2_output(immediate_field_extended_SL2)
164     );
165 
166     Adder Immediate(
167         .adder_input_1(pc_by_4),
168         .adder_input_2(immediate_field_extended_SL2),
169         .adder_output(branch_pc_by_4)
170     );
171 
172     Adder PC_Plus_4(
173         .adder_input_1(32'd4),
174         .adder_input_2(current_pc),
175         .adder_output(pc_by_4)
176     );
177 
```

```
178     MUX2x1 MUX_RF(
179         .mux_input_1(read_reg_2),
180         .mux_input_2(write_reg_input_2),
181         .mux_sel(reg_dst),
182         .mux_out(write_reg)
183     );
184
185     MUX2x1 MUX_ALU(
186         .mux_input_1(reg_file_read_data_2),
187         .mux_input_2(immediate_field_extended),
188         .mux_sel(alu_src),
189         .mux_out(alu_input_2)
190     );
191
192     MUX2x1 MUX_DM(
193         .mux_input_1(alu_res),
194         .mux_input_2(data_mem_read_data),
195         .mux_sel(mem_to_reg),
196         .mux_out(reg_file_write_data)
197     );
198
199     MUX2x1 MUX_branch_pc_by_4(
200         .mux_input_1(pc_by_4),
201         .mux_input_2(branch_pc_by_4),
202         .mux_sel(branch_sel),
203         .mux_out(branch_jump)
204     );
205
206     MUX2x1 MUX_PC(
207         .mux_input_1(branch_jump),
208         .mux_input_2(jump_address),
209         .mux_sel(jump),
210         .mux_out(next_pc)
211     );
212 endmodule
```

Top Module Figures

5.0 Synthesis and Resource Utilization

After completing the single-cycle MIPS processor, the system was successfully synthesized using Xilinx Vivado targeting the xc7s50csga324-1 FPGA device, where the RTL Verilog description was translated into FPGA hardware resources such as LUTs, registers, and memory structures. The synthesis results demonstrated efficient hardware utilization due to the modular and synthesis-friendly design methodology adopted throughout the project. Simple combinational modules such as multiplexers, adders, sign extension units, and shifting logic consumed minimal resources because Vivado optimized and merged much of the combinational logic internally.

The Arithmetic Logic Unit (ALU) utilized a moderate number of LUTs due to arithmetic and logical operations, while the Register File primarily consumed flip-flop resources because of its register-based storage implementation. Additionally, the Data Memory module was efficiently inferred using FPGA memory resources instead of large LUT-based logic implementation, significantly reducing overall resource consumption. The synthesis results confirmed the successful implementation of the processor datapath, control logic, branching mechanisms, memory operations, and instruction execution within a compact and efficient FPGA hardware design.

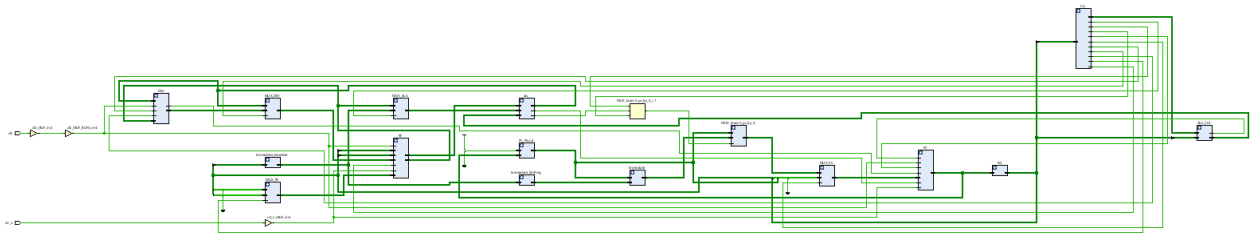

Figure 13: Synthesis Schematic (Not Expanded)

Name	Slice LUTs (32600)	Slice Registers (65200)	F7 Muxes (16300)	F8 Muxes (8150)	Bonded IOB (210)	BUFGCTRL (32)
Top_Module	1512	1024	512	128	2	1
alu (ALU)	166	0	0	0	0	0
DM (Data_Memory)	571	0	256	128	0	0
RF (Reg_File)	608	992	256	0	0	0

Figure 14: Synthesis Resource Utilization

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	inf	Worst Hold Slack (WHS):	inf	Worst Pulse Width Slack (WPWS):	NA
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	NA
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	NA
Total Number of Endpoints:	1056	Total Number of Endpoints:	1056	Total Number of Endpoints:	NA

There are no user specified timing constraints.

Figure 15: Synthesis Timing Report

6.0 Conclusion

This project successfully implemented a functional single-cycle MIPS processor using Verilog HDL. The processor supports multiple instruction formats and demonstrates correct execution behavior using a modular datapath and centralized control architecture.

The design provided practical experience in:

- RTL hardware design
- Processor architecture
- Verilog HDL coding
- Datapath implementation
- Control logic generation
- Simulation and debugging

The final architecture establishes a strong foundation for future extensions such as pipelined execution, hazard detection, forwarding logic, cache integration, and performance optimization techniques.

7.0 Future Work

Although the implemented processor successfully demonstrates the operation of a functional single-cycle MIPS architecture, several enhancements can be introduced in future development stages to improve performance and architectural efficiency. These improvements would extend the processor beyond basic single-cycle execution toward more advanced processor design methodologies commonly used in modern computing systems.

Future work primarily focuses on introducing pipelined execution, improving instruction throughput, and implementing mechanisms to handle pipeline hazards efficiently.

7.1 Pipelined Architecture Upgrade

One of the most significant future improvements for this project is upgrading the processor from a single-cycle architecture to a pipelined architecture. In the current implementation, each instruction completes all execution stages within one clock cycle, causing the clock period to be limited by the slowest instruction path.

A pipelined architecture divides instruction execution into multiple stages, allowing several instructions to execute simultaneously in different stages of the processor.

Typical MIPS pipeline stages include:

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execute (EX)
- Memory Access (MEM)
- Write Back (WB)

Implementing pipelining would require the introduction of:

- Pipeline registers between stages
- Stage synchronization mechanisms
- Control signal propagation
- Pipeline flushing and stalling logic

Transitioning toward a pipelined architecture would provide deeper understanding of modern processor design techniques and performance optimization strategies.

7.2 Hazard Detection and Forwarding

After implementing pipelining, additional mechanisms would be required to handle pipeline hazards that occur when multiple instructions interact during simultaneous execution.

The main categories of pipeline hazards include:

- Data hazards
- Control hazards
- Structural hazards

Implementing forwarding and hazard detection logic would greatly improve processor efficiency while providing valuable experience in advanced RTL architecture design and processor verification methodologies.

8.0 References

1. David A. Patterson and John L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, Morgan Kaufmann Publishers.
2. Morris Mano and Michael Ciletti, Digital Design, Pearson Education.
3. Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, Prentice Hall.
4. MIPS Technologies Inc., MIPS32 Architecture for Programmers.
5. IEEE Standard for Verilog Hardware Description Language (IEEE 1364).
6. Xilinx Vivado Design Suite Documentation.